

# InterfaceHive CI/CD Architecture

A comprehensive overview of our continuous integration and deployment pipeline

Code

Build

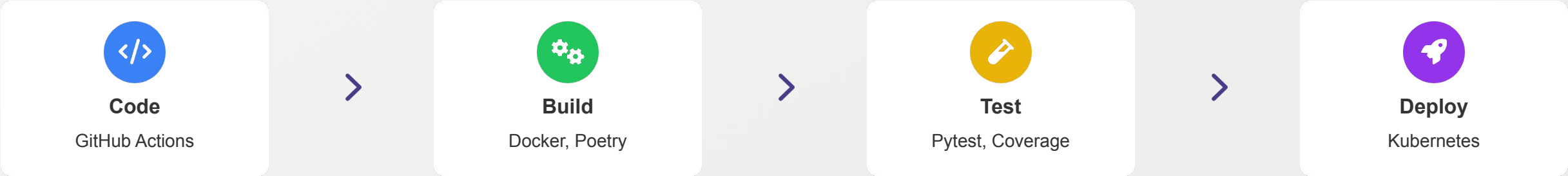
Test

Deploy

Monitor

# Pipeline Overview & Workflow

Multi-stage CI/CD pipeline architecture



## Development Workflow

- GitHub Actions automation
- Automated testing and build on PR merge
- Automatic deployment in dev-
- Postman test automation

## Production Workflow

- Manual approval required
- Blue-green deployment strategy
- Health check and automatic rollback
- Production-only secrets management

# Build & Testing Strategy

Automated build and testing strategy



## Docker Containerization

- Frontend: Node.js 18-alpine
- Backend: Python 3.11-slim
- Multi-stage build process



## Quality Analysis

- Code Coverage: pytest --cov
- Static Analysis: flake8, mypy
- Mutation Testing: pytest-mutpy



## Testing Framework

- Unit Tests: Python, React
- Integration Tests: PostgreSQL, Redis
- E2E Tests: Playwright

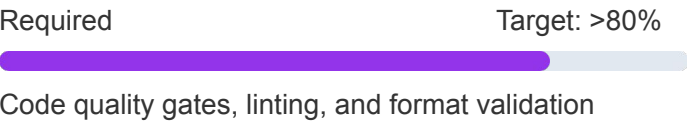


## Security Scanning

- Dependency Review: pip-audit
- Secret Detection: GitHub Secret Scanning
- Container Scanning: GitHub Container Registry



## Test Coverage & Quality



## Build & Test Workflow



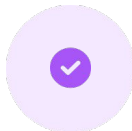
Code



Build



Test



Quality Gate

# Deployment & Infrastructure

Container orchestration through Kubernetes

## Infrastructure as Code

- ✓ **Kubernetes:** Cluster management and auto-scaling
- ✓ **Helm Charts:** Application packaging
- ✓ **Docker:** Manage container image
- ✓ **GitHub Actions:** CI/CD workflow

## Environment Configs

**dev-**  
Automatic deployment

**staging**  
Manual approval after QA

**production**  
Manual approval required

## Blue-Green Deployment

 Blue

 Green

- Zero-downtime releases
- Health checks
- Auto rollback

## Database Migrations

 Alembic



✓ Migration execution



 Service restart

# Monitoring & Quality Assurance

Continuous monitoring and quality assurance for production environment

## Monitoring Tools



### Prometheus

- Request rate and response times tracking
- Error rate and database performance monitoring
- Metrics provision for auto-scaling



### Grafana

- Dashboard creation and sharing
- Alert condition setup and management
- Data source integration (Prometheus)



### Alertmanager

- Alert aggregation and deduplication
- Alert notifications via Slack, Email, etc.
- Noise reduction through silence rules

## Health Checks & Automation



### Health Check Implementation



#### Liveness Probe

Failure detection for container restart



#### Readiness Probe

Conditions that delay service startup



#### Rolling Update Strategy

Strategy for 0% downtime deployment



### Automated Rollback Mechanism

- Automatic rollback on health check failure
- Blue-green deployment strategy
- Deployment status verification and automation
- Production-only secrets management

## Logging & Dashboard



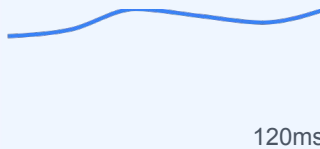
### Logging Strategy

- JSON format logging
- Contains service name, level, timestamp fields
- Error tracking through Sentry
- Log aggregation system integration

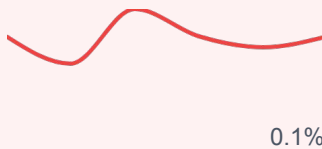


### Dashboard Preview

#### Response Time (p95)



#### Error Rate



#### Memory Usage



#### CPU Usage

